A Differentiable Hybrid Framework for Incompressible Flow Simulation, Control, and Inverse Design

---

Arno Jan Meijer

Independent – No Affiliation

Differential CFD-ML & Inverse Design

Abstract

This document presents a fully differentiable, structure-preserving framework for incompressible flow simulation, control, and inverse design, organized into three integrated components supported by comprehensive theoretical foundations, numerical implementations, transition roadmaps, and forward-looking extensions.

Part I establishes the foundation: a differentiable hybrid Navier–Stokes solver combining physics-based prediction, subgrid-scale closure, and multi-scale neural correction. A predictor step computes an intermediate velocity field, which is corrected by fine-scale and coarse-scale neural operators to enforce incompressibility without explicitly solving a pressure Poisson equation. Differentiable boundary conditions are applied via Brinkman penalization and masking operations.

Part II enhances the solver with advanced modeling capabilities. Sharp boundary recovery is achieved using signed distance functions to improve near-wall accuracy. Temporal modeling captures long-range dependencies through sequence-based operators, stabilizing long-horizon simulations. Adaptive mesh refinement concentrates resolution in regions of high flow activity, while latent multi-fidelity representations enable scalable high-dimensional modeling through compressed latent spaces.

Part III introduces closed-loop sensing and control. Differentiable virtual sensors provide targeted, interpretable feedback from flow and geometry. A flow actuation layer enables direct physical intervention through parameterized forcing mechanisms. Reinforcement-driven optimization formulates the system as a continuous control problem, where a differentiable policy maps states to actions, enabling long-term exploration beyond local gradient descent.

The framework is grounded in rigorous theoretical analysis addressing consistency, stability, and convergence, with numerical implementations provided for the baseline solver, inverse design automation, and latent-space extensions. Transition plans detail the stepwise development from a baseline vortex-shedding solver through latent acceleration to differentiable inverse design. Forward-looking extensions address scalability to three-dimensional flows, unstructured meshes,

and multi-GPU clusters using NVIDIA Warp and NVLink, alongside mitigation strategies for numerical stiffness, latent drift, spectral bias, and vanishing gradients.

All components are fully differentiable, supporting end-to-end gradient-based optimization for advanced inverse design applications. The unified framework balances physical accuracy, computational efficiency, and control flexibility, providing a novel approach for high-fidelity flow simulation and active flow management.

Keywords: Navier-Stokes, Differentiable Solver, Neural Networks, Multi-Scale Modeling, Active Flow Control, Reinforcement Learning, Inverse Design, Latent Space Modeling, Differentiable CFD, GPU Computing

Abstract 2

Part I: Differentiable Hybrid Navier–Stokes Solver with Multi-Scale Neural Correction 5

Part II: Active Flow Control and Enhanced Neural Flow Management 10

Part III: Active Sensing, Control, and Reinforcement-Driven Differentiable Flow Optimization 16

Appendix A: Theoretical Foundations, Consistency, Stability, and Convergence 21

Appendix B-I: Baseline Differentiable Hybrid Solver 29

Appendix B-II: Differentiable Inverse Design and Automated Training Framework 35

Appendix B-III: Latent Multi-Fidelity Modeling and Differentiable Control Integration 41

Appendix C-I: Transition Plan: Baseline Solver to Part I Completion 46

Appendix C-II: Transition Plan: Part I to Part II Completion 51

Appendix C-III: Transition Plan: Part II to Part III Completion 55

Appendix D - Towards a 3D Differentiable Hybrid CFD–ML Framework for High-Performance Super-Clusters 59

Appendix E - Mitigation Strategies for Numerical and Learning Challenges in a Differentiable Hybrid CFD–ML Solver 64

Part I: Differentiable Hybrid Navier–Stokes Solver with Multi-Scale Neural Correction

Purpose

This document presents a fully differentiable, structure-preserving solver for incompressible flow. The method combines physics-based prediction, subgrid-scale (SGS) closure, multi-scale neural correction, and differentiable boundary enforcement to enable stable forward simulation and end-to-end inverse design.

---

1. Governing Equations

The flow is governed by the incompressible Navier–Stokes equations with Brinkman penalization and subgrid-scale closure:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u} + \nabla \cdot \boldsymbol{\tau}_{\text{SGS}} - \frac{\chi(\mathbf{x}; \boldsymbol{\theta})}{\epsilon}\mathbf{u}$$

$$\nabla \cdot \mathbf{u} = 0$$

Where:

$\mathbf{u}$	→	velocity field
p	→	pressure
ν	→	kinematic viscosity
$\boldsymbol{\tau}_{\text{SGS}}$	→	subgrid-scale stress tensor
$\chi(\mathbf{x}; \boldsymbol{\theta})$	→	differentiable solid mask
ϵ	→	Brinkman penalization parameter

The Brinkman term introduces a distributed momentum sink, enabling smooth and differentiable representation of solid boundaries.

---

2. Spatial Discretization

A uniform Cartesian grid is employed:

$$(x_i, y_j), \Delta x = \Delta y = h$$

Discrete operators:

Advection:	second-order central differences
Diffusion:	second-order Laplacian
Divergence:	central difference operator
Vorticity:	$\omega = \nabla \times \mathbf{u}$

No explicit upwinding is used; numerical stability is maintained through SGS modeling and neural correction.

---

3. Predictor Step (Physics + SGS)

An intermediate velocity field is computed via explicit time integration:

$$\mathbf{u}^* = \mathbf{u}^n + \Delta t(-(\mathbf{u}^n \cdot \nabla)\mathbf{u}^n + \nu\nabla^2\mathbf{u}^n + \nabla \cdot \boldsymbol{\tau}_{\text{SGS}})$$

The intermediate field $\mathbf{u}^*$ is generally non-solenoidal. No explicit pressure projection is performed at this stage.

---

4. Emergent Feature Extraction

A local feature vector is constructed from the intermediate field:

$$\mathbf{F} = \begin{bmatrix} \nabla \cdot \mathbf{u}^* \\ \omega^* \\ |\,\mathbf{u}^*\,| \\ \|\,S^*\,\| \\ \chi(\mathbf{x}; \boldsymbol{\theta}) \end{bmatrix}$$

Where the strain-rate tensor is defined as:

$$S = \frac{1}{2}(\nabla\mathbf{u} + \nabla\mathbf{u}^T)$$

These features encode local constraint violations, flow structure, and geometric context.

5. Multi-Scale Neural Correction

The intermediate velocity field is corrected using a combination of fine-scale and coarse-scale neural operators.

5.1. Fine-Scale Network

The fine-scale network operates on local features:

$$\mathbf{u}_{\text{corr,local}} = \mathcal{N}_{\text{fine}}(\mathbf{F})$$

This component targets high-frequency flow structures, including small-scale vortices and localized divergence errors. The correction is constrained to be approximately divergence-free through architectural design or loss enforcement.

5.2. Coarse-Scale Network

A coarse representation of the features is constructed using a restriction operator $\mathcal{R}$:

$$\mathbf{u}_{\text{corr,global}} = \mathcal{N}_{\text{coarse}}(\mathcal{R}(\mathbf{F}))$$

This component approximates global pressure effects by redistributing divergence over long spatial scales. The resulting correction is mapped back to the fine grid via a prolongation operator $\mathcal{P}_{\uparrow}$.

5.3. Combined Update

The corrected velocity field is given by:

$$\mathbf{u}^{\text{corr}} = \mathbf{u}^{*} + \mathbf{u}_{\text{corr,local}} + \mathcal{P}_{\uparrow}(\mathbf{u}_{\text{corr,global}})$$

This formulation enables simultaneous correction of local and global flow inconsistencies without explicitly solving a pressure Poisson equation.

---

6. Differentiable Boundary Enforcement

Boundary conditions are enforced using a differentiable masking operation:

$$\mathbf{u}^{\text{final}} = (1 - \chi(\mathbf{x}; \boldsymbol{\theta}))\mathbf{u}^{\text{corr}}$$

This operation suppresses velocity within solid regions while maintaining differentiability with respect to geometric parameters.

The masking step complements the Brinkman penalization by providing stronger enforcement of near-wall behaviour.

---

7. Loss Function

The training objective combines physical consistency and design constraints:

$$\mathcal{L} = \alpha \parallel \nabla \cdot \mathbf{u}^{\text{final}} \parallel^2 + \beta \parallel \omega^{\text{final}} - \omega_{\text{target}} \parallel^2 + \lambda \mathcal{L}_{\text{design}} + \eta \mathcal{L}_{\text{global}}$$

Where:

$\alpha \parallel \nabla \cdot \mathbf{u}^{\text{final}} \parallel^2$	→	the first term enforces incompressibility
$\beta \parallel \omega^{\text{final}} - \omega_{\text{target}} \parallel^2$	→	the second term preserves target flow structures
$\lambda \mathcal{L}_{\text{design}}$	→	application-specific objective (e.g., drag, lift)
$\eta \mathcal{L}_{\text{global}}$	→	low-wavenumber stabilization

---

8. Time-Stepping Algorithm

For each timestep:

STEP	OPERATION
1	*Compute subgrid-scale stress*
2	*Perform predictor step to obtain* $\mathbf{u}^*$

3	*Extract feature field* $\mathbf{F}$
4	*Apply fine-scale neural correction*
5	*Apply coarse-scale neural correction*
6	*Combine corrections to obtain* $\mathbf{u}^{\text{corr}}$
7	*Apply differentiable boundary mask*
8	*Advance solution to next timestep*

All operations are differentiable, enabling end-to-end gradient propagation.

---

9. Multi-Scale Operator Structure

$$\mathbf{u}^n \rightarrow \text{Predictor} \rightarrow \mathbf{u}^* \rightarrow \begin{cases} \text{Fine-scale NN} \\ \text{Coarse-scale NN} \end{cases} \rightarrow \mathbf{u}^{\text{corr}} \rightarrow \text{Masking} \rightarrow \mathbf{u}^{\text{final}}$$

The fine-scale network resolves local flow structures, while the coarse-scale network approximates global pressure redistribution.

---

10. Advantages

- Fully differentiable solver suitable for inverse design
- Avoids explicit pressure Poisson solves
- Structure-preserving through divergence-aware corrections
- Multi-scale formulation captures both local and global flow behaviour
- Differentiable boundary representation via Brinkman penalization and masking
- Compatible with modern *autodiff* frameworks

---

11. Limitations

- Global pressure effects are approximated rather than explicitly enforced
- Soft boundary representation introduces limited interface smearing
- Training stability depends on balancing neural and physical terms
- Multi-scale architecture increases implementation complexity

Part II: Active Flow Control and Enhanced Neural Flow Management

Purpose

This document extends the differentiable hybrid Navier–Stokes solver by incorporating sharp boundary recovery, temporal modeling, adaptive spatial resolution, and latent multi-fidelity representations. These enhancements improve accuracy, stability, and scalability, enabling high-fidelity inverse design and long-horizon flow simulation.

---

1. Sharp Boundary Recovery

1.1. Motivation

Brinkman penalization provides a smooth and differentiable representation of solid boundaries but introduces diffuse interfaces. While suitable for optimization, this soft representation can degrade accuracy near walls, particularly in applications requiring precise surface interactions such as aerodynamic or hydrodynamic design.

1.2. Formulation

Sharp interface recovery is achieved using a Signed Distance Function (SDF) representation of geometry:

$$\phi(\mathbf{x}; \boldsymbol{\theta})$$

A differentiable masking function is constructed using a smoothed Heaviside approximation:

$$\mathbf{u}^{\text{final}} = \mathbf{u}^{\text{corr}} \cdot (1 - H_\epsilon(\phi(\mathbf{x}; \boldsymbol{\theta})))$$

Where:

$\phi(\mathbf{x}; \boldsymbol{\theta})$ → signed distance to the boundary

H_ϵ → smooth approximation of the Heaviside function

This formulation enforces a sharp transition at the fluid–solid interface while maintaining differentiability with respect to geometric parameters.

1.3. Interpretation

The SDF-based masking acts as a strong boundary enforcement mechanism, complementing the Brinkman penalization introduced in Part I. Together, these provide both smooth optimization behaviour and accurate near-wall resolution.

---

2. Temporal Modeling via Sequence Operators

2.1. Motivation

The coarse-scale neural correction introduced in Part I operates on instantaneous features and captures only short-range temporal effects. However, many flow phenomena—such as vortex shedding, wake interaction, and recirculation—exhibit long-term temporal dependencies.

2.2. Formulation

Temporal correlations are incorporated using a sequence-based operator:

$$\mathbf{u}^{t}_{\text{corr,global}} = \psi_{\text{temporal}}(\mathbf{F}_{t-k:t})$$

Where:

$\mathbf{F}_{t-k:t}$	→	sequence of feature fields over the previous
k	→	timesteps
ψ_{temporal}	→	sequence model (e.g., transformer or graph-based operator)

The temporal operator replaces or augments the coarse-scale network, enabling global corrections informed by historical flow evolution.

2.3. Properties

- Captures long-range temporal dependencies
- Stabilizes long-horizon simulations
- Improves global divergence redistribution consistency

---

3. Adaptive Mesh Refinement (AMR)

3.1. Motivation

Uniform grids allocate computational resources uniformly across the domain, regardless of flow complexity. In practice, high resolution is required only in localized regions such as boundary layers, shear layers, and vortical structures.

3.2. Formulation

A spatially adaptive grid is constructed using a differentiable refinement criterion:

$$h(\mathbf{x}) \sim f(|\nabla \cdot \mathbf{u}|,\ |\omega|,\ \|\nabla \mathbf{u}\|)$$

Where $h(\mathbf{x})$ denotes local grid spacing.

Refinement is implemented using hierarchical structures (e.g., quadtrees in 2D or octrees in 3D) with differentiable interpolation and restriction operators.

3.3. Properties

- Concentrates resolution in regions of high flow activity
- Reduces computational cost in low-gradient regions
- Preserves differentiability for gradient-based optimization

---

4. Latent Multi-Fidelity Representation

4.1. Motivation

Direct neural correction in physical space becomes computationally expensive for high-resolution or three-dimensional flows. Additionally, many regions exhibit redundant or low-information dynamics.

4.2. Formulation

A latent representation of the flow is introduced:

$$\mathbf{z} = \mathcal{E}(\mathbf{F}), \mathbf{u}_{\text{corr}} = \mathcal{D}(\mathbf{z})$$

Where:

$\mathcal{E}$ → encoder mapping physical features to latent space

$\mathcal{D}$ → decoder reconstructing velocity corrections

The latent operator may incorporate diffusion-based or operator-learning techniques to model flow evolution in a compressed space.

4.3. Training Objectives

Latent representations are trained to preserve:

- Divergence-free structure
- Vorticity dynamics
- Design-relevant flow features

4.4. Properties

- Reduces computational complexity
- Enables large-scale simulations on limited hardware
- Supports hierarchical and multi-fidelity training strategies

---

5. Extended Loss Function

The loss function is augmented to account for temporal consistency, latent fidelity, and adaptive resolution:

$$\mathcal{L} = \alpha \parallel \nabla \cdot \mathbf{u}^{\text{final}} \parallel^2 + \beta \parallel \omega^{\text{final}} - \omega_{\text{target}} \parallel^2 + \lambda \mathcal{L}_{\text{design}} + \eta \mathcal{L}_{\text{global}} + \gamma \mathcal{L}_{\text{temporal}} + \delta \mathcal{L}_{\text{latent}} + \kappa \mathcal{L}_{\text{AMR}}$$

Where:

$\gamma \mathcal{L}_{\text{temporal}}$	→	penalizes inconsistency across time sequences
$\delta \mathcal{L}_{\text{latent}}$	→	enforces reconstruction fidelity in latent space
$\kappa \mathcal{L}_{\text{AMR}}$	→	regularizes grid adaptation and interpolation consistency

---

6. Integrated Multi-Scale Flow Architecture

$\mathbf{u}^n$ → Predictor → $\mathbf{u}^*$ → Fine-Scale Correction → Temporal/Coarse Correction → Latent Mapping → Adaptive Refinement → Sharp Boundary Enforcement → $\mathbf{u}^{\text{final}}$

This architecture integrates spatial, temporal, and latent representations within a fully differentiable framework.

---

7. Advantages

- Sharp interface recovery improves near-wall accuracy
- Temporal modeling enables stable long-horizon simulations
- Adaptive mesh refinement reduces computational overhead
- Latent representations enable scalable high-dimensional modeling
- Fully differentiable, supporting gradient-based inverse design
- Compatible with modern autodiff and accelerator frameworks

---

8. Limitations

- Increased architectural and implementation complexity
- Temporal models require careful selection of sequence length
- Latent compression may introduce reconstruction errors
- Sharp boundary enforcement can introduce numerical stiffness
- Training stability depends on appropriate weighting of loss components

---

9. Position Within the Framework

Part II extends the baseline solver introduced in Part I by introducing:

- Strong boundary enforcement via SDF-based masking
- Long-term temporal modeling
- Adaptive spatial resolution
- Latent multi-fidelity representations

Part III: Active Sensing, Control, and Reinforcement-Driven Differentiable Flow Optimization

Purpose

This document introduces a closed-loop extension of the differentiable hybrid Navier–Stokes framework by incorporating active sensing, flow actuation, and reinforcement-driven optimization. The formulation enables adaptive, self-correcting flow evolution and supports long-horizon, high-fidelity inverse design through integrated control of both flow physics and geometry.

---

1. Active Flow Sensing

1.1. Motivation

Feature-based representations (e.g., divergence, vorticity, strain) provide implicit information about flow behaviour but may not sufficiently capture critical localized phenomena such as separation, wake instability, or wall-bounded effects. Direct sensing enables targeted observation of physically meaningful quantities at strategically important locations.

1.2. Formulation

Differentiable virtual sensors are defined as mappings from the flow field and geometry to observable quantities:

$$\mathbf{s}(\mathbf{x}, t) = \mathcal{S}(\mathbf{u}^{\text{final}}(\mathbf{x}, t), \phi(\mathbf{x}; \boldsymbol{\theta}))$$

Where:

$\mathcal{S}$ → differentiable sensing operator

$\phi(\mathbf{x}; \boldsymbol{\theta})$ → signed distance function representing geometry

Sensor outputs may include:

- Wall shear stress
- Surface pressure
- Local vorticity or circulation
- Wake intensity metrics

1.3. Properties

- Provides localized, physically interpretable feedback

- Enables targeted correction and control of critical flow regions
- Fully differentiable with respect to both flow state and geometry

---

2. Flow Actuation Layer

2.1. Motivation

Neural corrections modify the flow indirectly through learned adjustments to velocity fields. In high-Reynolds-number or strongly separated flows, direct physical intervention is often required to achieve desired flow characteristics.

2.2. Governing Equations with Actuation

The momentum equation is augmented with a differentiable actuation term:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u} + \nabla \cdot \boldsymbol{\tau}_{\text{SGS}} - \frac{\chi}{\epsilon}\mathbf{u} + \mathbf{f}_{\text{act}}$$

Where $\mathbf{f}_{\text{act}}$ denotes the actuation field.

2.3. Actuation Modeling

The actuation field is parameterized as a differentiable function:

$$\mathbf{f}_{\text{act}} = \mathcal{A}(\mathbf{x}, t; \boldsymbol{\psi})$$

Where $\boldsymbol{\psi}$ represents control parameters or neural outputs.

Possible actuation mechanisms include:

- Surface blowing or suction
- Synthetic jet forcing
- Distributed body-force approximations of vortex generators

2.4. Properties

- Enables direct manipulation of flow structures
- Allows optimization of physical control strategies
- Fully compatible with gradient-based optimization

---

3. Reinforcement-Driven Optimization

3.1. Motivation

Gradient-based optimization is effective for local improvements but may converge to suboptimal solutions in highly non-convex design spaces. Reinforcement learning introduces a framework for long-horizon optimization and exploration of complex control strategies.

3.2. Formulation

The system is formulated as a continuous control problem:

State:

$$\mathbf{s}_t = \{\mathbf{F}_t, \mathbf{s}(\mathbf{x}, t)\}$$

where $\mathbf{F}_t$ denotes emergent flow features and $\mathbf{s}(\mathbf{x}, t)$ sensor outputs.

Action: $\mathbf{a}_t = \{\mathbf{f}_{\text{act}}, \boldsymbol{\theta}\}$ including both actuation and geometry updates.

Reward: $R_t = -\mathcal{L}_{\text{design}}(\mathbf{u}^{\text{final}}, \omega_{\text{target}}, \phi)$

3.3. Policy Optimization

A differentiable policy $\pi_{\boldsymbol{\psi}}$ maps states to actions:

$$\mathbf{a}_t = \pi_{\boldsymbol{\psi}}(\mathbf{s}_t)$$

Optimization is performed using policy gradient methods, leveraging automatic differentiation through the full solver pipeline.

3.4. Properties

- Enables long-term optimization across multiple timesteps
- Encourages exploration beyond local gradient descent
- Integrates naturally with differentiable simulation

---

4. Combined Objective Function

The total loss incorporates physical consistency, design objectives, and control regularization:

$$\mathcal{L}_{\text{total}} = \alpha \parallel \nabla \cdot \mathbf{u}^{\text{final}} \parallel^2 + \beta \parallel \omega^{\text{final}} - \omega_{\text{target}} \parallel^2 + \lambda \mathcal{L}_{\text{design}} + \eta \mathcal{L}_{\text{global}} + \gamma \mathcal{L}_{\text{temporal}} + \delta \mathcal{L}_{\text{latent}} + \kappa \mathcal{L}_{\text{AMR}} + \mu \mathcal{L}_{\text{actuation}}$$

Where $\mu \mathcal{L}_{\text{actuation}}$ penalizes excessive or non-physical control inputs.

This formulation ensures that control actions remain physically plausible while achieving design objectives.

---

5. Closed-Loop Flow Control Architecture

$\mathbf{u}^n \rightarrow$ Predictor $\rightarrow \mathbf{u}^* \rightarrow$ Neural Corrections $\rightarrow$

Latent Representation $\rightarrow$ Adaptive Refinement $\rightarrow$

Sensors $\rightarrow$ Policy (Control) $\rightarrow$

Actuation $\rightarrow$ Boundary Enforcement $\rightarrow$

$\rightarrow \mathbf{u}^{\text{final}}$

This architecture forms a fully differentiable feedback loop in which sensing, control, and flow evolution are jointly optimized.

---

6. Advantages

- Closed-loop control enables precise manipulation of complex flow structures
- Differentiable sensing provides targeted and interpretable feedback
- Actuation layer enables physically realizable flow control strategies
- Reinforcement learning enables exploration of non-trivial design solutions
- Fully integrated with differentiable solver framework
- Supports real-time and adaptive inverse design in complex flows

---

7. Limitations

- High implementation complexity due to integration of sensing, control, and learning components
- Performance depends on sensor placement and observability
- Actuation may introduce numerical stiffness or stability challenges
- Reinforcement learning requires careful reward shaping and tuning
- Training cost increases with system dimensionality and control horizon

---

8. Framework Summary

The complete framework consists of three integrated components:

- A differentiable hybrid Navier–Stokes solver with multi-scale neural correction
- Enhanced modeling through sharp boundaries, temporal dynamics, adaptive resolution, and latent representations
- Closed-loop sensing and control with reinforcement-driven optimization
- Together, these components define a unified approach to high-fidelity, differentiable, and controllable flow simulation suitable for advanced inverse design applications.

Appendix A: Theoretical Foundations, Consistency, Stability, and Convergence

---

A.1. Scope and Objectives

This appendix provides the theoretical grounding for the differentiable hybrid Navier–Stokes framework. It formalizes:

- Consistency with the incompressible Navier–Stokes equations
- Interpretation of neural operators as approximate projection mechanisms
- Stability considerations under multi-scale neural correction
- Convergence intuition and limiting behaviour

The goal is not to provide strict proofs, but to establish a rigorous and physically grounded justification of the method.

A.2. Discrete Governing System

The semi-discrete form of the governing equations can be written as:

$$\frac{d\mathbf{u}}{dt} = \mathcal{N}(\mathbf{u}) - \nabla p - \frac{\chi}{\epsilon}\mathbf{u}$$

where:

$$\mathcal{N}(\mathbf{u}) = -(\mathbf{u}\cdot\nabla)\mathbf{u} + \nu\nabla^2\mathbf{u} + \nabla\cdot\boldsymbol{\tau}_{\mathrm{SGS}}$$

The incompressibility constraint is:

$$\nabla\cdot\mathbf{u} = 0$$

A.3. Classical Projection Formulation

In traditional fractional-step methods, incompressibility is enforced via a projection step:

$$\mathbf{u}^{n+1} = \mathbf{u}^* - \nabla\phi$$

Where ϕ satisfies the Poisson equation:

$$\nabla^2\phi = \nabla \cdot \mathbf{u}^*$$

This defines the Leray projection operator:

$$\mathcal{P}(\mathbf{u}^*) = \mathbf{u}^* - \nabla(\nabla^{-2}\nabla \cdot \mathbf{u}^*)$$

A.4. Neural Approximation of the Projection Operator

The proposed framework replaces the explicit projection with a learned operator:

$$\mathbf{u}^{n+1} = \mathbf{u}^* + \mathcal{N}_{\text{corr}}(\mathbf{F})$$

where:

$$\mathcal{N}_{\text{corr}} = \mathcal{N}_{\text{fine}} + \mathcal{P}_{\uparrow} \circ \mathcal{N}_{\text{coarse}} \circ \mathcal{R}$$

A.4.1. Interpretation

The neural correction approximates:

$$\mathcal{N}_{\text{corr}}(\mathbf{F}) \approx -\nabla(\nabla^{-2}\nabla \cdot \mathbf{u}^*)$$

Thus, the full update approximates:

$$\mathbf{u}^{n+1} \approx \mathcal{P}(\mathbf{u}^*)$$

A.4.2. Multi-Scale Decomposition

The decomposition into fine and coarse operators can be interpreted as:

- Fine-scale operator: approximation of high-frequency components of the projection

- Coarse-scale operator: approximation of low-frequency (global) pressure modes

This aligns with multigrid interpretations of the Poisson solve.

A.5. Consistency

A.5.1. Spatial and Temporal Consistency

Given:

- Second-order spatial discretization
- Explicit time integration

The predictor step is consistent with the Navier–Stokes equations:

$$\mathbf{u}^* = \mathbf{u}(t + \Delta t) + \mathcal{O}(\Delta t^2, h^2)$$

A.5.2. Projection Consistency

If the neural operator satisfies:

$$\mathcal{N}_{\text{corr}}(\mathbf{F}) \to -\nabla(\nabla^{-2}\nabla \cdot \mathbf{u}^*)$$

then:

$$\nabla \cdot \mathbf{u}^{n+1} \to 0$$

Thus, in the limit of perfect learning:

- The method recovers the classical projection scheme
- The solution converges to the incompressible Navier–Stokes solution

---

A.5.3. Boundary Consistency

The combined Brinkman penalization and SDF masking yields:

$$\mathbf{u} \to 0 \text{ as } \chi \to 1$$

As $\epsilon \to 0$, and the SDF sharpens:

- The solution approaches a no-slip boundary condition
- Interface thickness tends to zero

A.6. Stability Considerations

A.6.1. Energy Evolution

Define kinetic energy:

$$E = \frac{1}{2} \parallel \mathbf{u} \parallel^2$$

The physical system satisfies:

$$\frac{dE}{dt} = -\nu \parallel \nabla \mathbf{u} \parallel^2 - \text{dissipation from SGS} - \frac{1}{\epsilon} \parallel \chi \mathbf{u} \parallel^2$$

A.6.2. Neural Correction Stability Requirement

For stability, the neural correction must satisfy:

$$\langle \mathbf{u}, \mathcal{N}_{\text{corr}} \rangle \leq C \parallel \nabla \cdot \mathbf{u} \parallel^2$$

This ensures:

- No artificial energy injection at large scales
- Corrections act primarily to remove divergence

A.6.3. Role of SGS Model

The SGS stress provides controlled dissipation:

$$\nabla \cdot \boldsymbol{\tau}_{\text{SGS}} \approx -\nu_t \nabla^2 \mathbf{u}$$

Where ν_t is an effective turbulent viscosity.

This compensates for:

- Non-dissipative central differencing
- Potential instabilities from neural corrections

A.6.4. Temporal Operator Stability

The temporal model introduces a memory operator:

$$\mathbf{u}_{\text{corr}}^{t} = \psi(\mathbf{F}_{t-k:t})$$

Stability requires bounded amplification:

$$\| \psi \| \leq 1 + \mathcal{O}(\Delta t)$$

to avoid exponential growth over long horizons.

A.7. Convergence Intuition

A.7.1. Grid Convergence

As $h \to 0$:

- Discretization error vanishes
- Neural operator approximates continuous projection

Thus:

$$\mathbf{u}_h \to \mathbf{u}$$

A.7.2. Operator Convergence

If neural operators are universal approximators:

$$\mathcal{N}_{\text{corr}} \to \mathcal{P}$$

then the scheme converges to the classical incompressible solution.

A.7.3. Multi-Scale Convergence

The restriction/prolongation framework ensures:

- Coarse operator captures global modes
- Fine operator captures local corrections

This mimics multigrid convergence behaviour:

$$\text{Error} = \text{high-frequency} + \text{low-frequency}$$

Both components are reduced per iteration.

A.7.4. Latent Space Convergence

Let:

$$\mathbf{z} = \mathcal{E}(\mathbf{F})$$

If encoder-decoder reconstruction satisfies:

$$\| \mathbf{F} - \mathcal{D}(\mathcal{E}(\mathbf{F})) \| \to 0$$

then latent evolution preserves physical fidelity.

A.8. Relation to Classical Methods

The proposed framework can be interpreted as:

Classical Component	Proposed Equivalent
Pressure Poisson solve	Coarse neural operator
Projection step	Neural correction
Multigrid solver	Multi-scale NN
Boundary conditions	Brinkman + SDF masking
Turbulence model	SGS + learned correction

A.9. Differentiability and Optimization

All components are differentiable:

- Spatial operators: linear and differentiable
- Neural operators: differentiable by construction
- Boundary representation: SDF-based
- Actuation and sensing: explicit differentiable mappings

Thus:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} \text{ exists and is computable}$$

This enables:

- Gradient-based inverse design
- End-to-end training
- Coupled optimization of flow, geometry, and control

A.10. Summary

The framework is theoretically grounded as:

1. A consistent discretization of incompressible Navier–Stokes
2. A neural approximation of the Leray projection operator
3. A multi-scale correction mechanism analogous to multigrid methods
4. A stable system when constrained by SGS dissipation and bounded neural operators
5. A convergent scheme under refinement and sufficient model capacity

This establishes a unified interpretation of the method as a learned, differentiable, multi-scale projection framework for incompressible flow.

Appendix B-I: Baseline Differentiable Hybrid Solver

B-I.1 Overview

This appendix presents the baseline numerical realization of a differentiable hybrid Navier–Stokes solver with multi-scale neural correction and differentiable boundary enforcement.

The formulation integrates:

- Physics-based predictor (advection–diffusion + SGS closure)
- Multi-scale neural correction operators
- Implicit divergence control via learned projection
- Differentiable boundary treatment using Brinkman penalization

The implementation is constructed using JAX and Equinox, ensuring compatibility with automatic differentiation and accelerator-based execution.

B-I.2 Grid and Discrete Operators

B-I.2.1 Cartesian Grid Definition

A uniform Cartesian grid is defined as:

```
python
class Grid2D:
    def __init__(self, nx, ny, dx, dy):
        self.nx, self.ny = nx, ny
        self.dx, self.dy = dx, dy
        self.x = jnp.linspace(0, nx*dx, nx)
        self.y = jnp.linspace(0, ny*dy, ny)
```

B-I.2.2 Discrete Differential Operators

Second-order central differencing is used:

```python
def central_diff_x(self, u):
    return (jnp.roll(u, -1, axis=0) - jnp.roll(u, 1, axis=0)) /
(2*self.dx)

def central_diff_y(self, u):
    return (jnp.roll(u, -1, axis=1) - jnp.roll(u, 1, axis=1)) /
(2*self.dy)

def laplacian(self, u):
    return (
        (jnp.roll(u, -1, 0) - 2*u + jnp.roll(u, 1, 0)) / self.dx**2 +
        (jnp.roll(u, -1, 1) - 2*u + jnp.roll(u, 1, 1)) / self.dy**2
    )
```

These operators are:

- Linear
- Translation-invariant
- Fully differentiable

and form the foundation of the physics-based predictor.

B-I.3 Subgrid-Scale Closure

B-I.3.1 SGS Stress Model

A simplified Germano-inspired model is employed:

```python
def compute_sgs_stress(u, grid, nu_sgs):
    Sx = 0.5 * (grid.central_diff_x(u[0]) + grid.central_diff_y(u[0]))
    Sy = 0.5 * (grid.central_diff_x(u[1]) + grid.central_diff_y(u[1]))
    S_mag = jnp.sqrt(2 * (Sx**2 + Sy**2))
    tau = -2 * nu_sgs * S_mag
    return tau
```

B-I.3.2 Role in the Solver

The SGS term provides:

- Controlled dissipation
- Stabilization of central differencing

- Separation of physical dissipation from constraint enforcement

B-I.4 Multi-Scale Neural Correction

B-I.4.1 Fine-Scale Network (Local Correction)

```
python
class FineScaleNN(eqx.Module):
    layers: list

    def __init__(self, key, input_dim, hidden_dim=64, output_dim=2):
        keys = jax.random.split(key, 3)
        self.layers = [
            eqx.nn.Linear(input_dim, hidden_dim, key=keys[0]),
            eqx.nn.Linear(hidden_dim, hidden_dim, key=keys[1]),
            eqx.nn.Linear(hidden_dim, output_dim, key=keys[2]),
        ]

    def __call__(self, x, grid):
        h = jax.nn.relu(self.layers[0](x))
        h = jax.nn.relu(self.layers[1](h))
        out = self.layers[2](h)

        u_corr = jnp.stack([
            grid.central_diff_y(out[..., 0]),
            -grid.central_diff_x(out[..., 1])
        ], axis=-1)

        return u_corr
```

B-I.4.2 Coarse-Scale Network (Global Correction)

python

```
class CoarseScaleNN(eqx.Module):
    layers: list

    def __init__(self, key, input_dim, hidden_dim=64, output_dim=2):
        keys = jax.random.split(key, 3)
        self.layers = [
            eqx.nn.Linear(input_dim, hidden_dim, key=keys[0]),
            eqx.nn.Linear(hidden_dim, hidden_dim, key=keys[1]),
            eqx.nn.Linear(hidden_dim, output_dim, key=keys[2]),
        ]

    def __call__(self, x, grid):
        h = jax.nn.relu(self.layers[0](x))
        h = jax.nn.relu(self.layers[1](h))
        out = self.layers[2](h)

        u_corr = jnp.stack([
            grid.central_diff_y(out[..., 0]),
            -grid.central_diff_x(out[..., 1])
        ], axis=-1)

        return u_corr
```

B-I.4.3 Divergence-Free Construction

Both networks enforce incompressibility via:

- Streamfunction-like formulation
- Curl-based reconstruction

This removes the need for an explicit pressure Poisson solve.

B-I.5 Time Integration and Solver Step

B-I.5.1 Predictor Step

```
        python
        adv_x = u[0]*grid.central_diff_x(u[0]) +
u[1]*grid.central_diff_y(u[0])
        adv_y = u[0]*grid.central_diff_x(u[1]) +
u[1]*grid.central_diff_y(u[1])

        diff = grid.laplacian(u)

        u_star = u + dt * (-jnp.array([adv_x, adv_y]) + diff + sgs_stress)
```

B-I.5.2 Feature Extraction

```
        python
        div = grid.central_diff_x(u_star[0]) + grid.central_diff_y(u_star[1])
        omega = grid.central_diff_x(u_star[1]) -
grid.central_diff_y(u_star[0])

        F = jnp.stack([
            div,
            omega,
            jnp.linalg.norm(u_star, axis=0),
            mask
        ], axis=-1)
```

B-I.5.3 Neural Correction and Update

```
        python
        u_corr = u_star + fine_nn(F, grid) + coarse_nn(F, grid)
```

B-I.5.4 Differentiable Boundary Enforcement

```
        python
        u_final = (1 - mask) * u_corr
```

This corresponds to Brinkman-type penalization in differentiable form.

B-I.5.5 Full Timestep Operator

```
python
def timestep(u, dt, grid, sgs_stress, fine_nn, coarse_nn, mask):

    adv_x = u[0]*grid.central_diff_x(u[0]) +
u[1]*grid.central_diff_y(u[0])
    adv_y = u[0]*grid.central_diff_x(u[1]) +
u[1]*grid.central_diff_y(u[1])

    diff = grid.laplacian(u)

    u_star = u + dt * (-jnp.array([adv_x, adv_y]) + diff + sgs_stress)

    div = grid.central_diff_x(u_star[0]) +
grid.central_diff_y(u_star[1])
    omega = grid.central_diff_x(u_star[1]) -
grid.central_diff_y(u_star[0])

    F = jnp.stack([div, omega, jnp.linalg.norm(u_star, axis=0), mask],
axis=-1)

    u_corr = u_star + fine_nn(F, grid) + coarse_nn(F, grid)

    return (1 - mask) * u_corr
```

B-I.6 Loss Function and Optimization

B-I.6.1 Loss Definition

```
python
def loss_fn(u_final, u_target, omega_target, grid, alpha, beta,
lambda_design):

    div = grid.central_diff_x(u_final[0]) +
grid.central_diff_y(u_final[1])
    omega = grid.central_diff_x(u_final[1]) -
grid.central_diff_y(u_final[0])

    return (
        alpha * jnp.mean(div**2) +
        beta * jnp.mean((omega - omega_target)**2) +
        lambda_design * jnp.mean((u_final - u_target)**2)
    )
```

B-I.6.2 Optimization Strategy

- Gradient-based optimization via JAX autodiff
- Compatible with Adam or second-order optimizers
- Supports rollout-based and single-step supervision

B-I.7 Implementation Notes

- Fully differentiable end-to-end
- Compatible with ***jit***, ***vmap***, and ***pmap***

- Neural correction replaces classical projection step
- Brinkman masking enables smooth geometry coupling
- Modular structure allows direct extension to Parts II and III

B-I.8 Summary

This appendix defines the baseline computational operator consisting of:

- Physics-based predictor
- SGS stabilization
- Multi-scale neural correction
- Differentiable boundary enforcement
- This forms the foundational building block upon which:
- Sharp boundary methods (Appendix B2)
- Temporal and adaptive modeling (Appendix B2)
- Latent and control frameworks (Appendix B3)
- Inverse design and training systems (Appendix B-II)

are constructed.

Appendix B-II: Differentiable Inverse Design and Automated Training Framework

---

B-II.1 Overview

This appendix formalizes the inverse design and automated training framework built on top of the differentiable hybrid Navier–Stokes solver. The objective is to enable:

- End-to-end optimization of geometry via differentiable Signed Distance Functions (SDFs)
- Coupled optimization of flow, control, and geometry
- Automated hyperparameter tuning and multi-fidelity training
- Scalable training for high-dimensional and long-horizon flow problems
- All components are fully differentiable and compatible with JAX-based automatic differentiation.

B-II.2 Differentiable Geometry Representation

B-II.2.1 Signed Distance Function Parameterization

Geometry is represented using a differentiable Signed Distance Function:

$$\phi(\mathbf{x}; \boldsymbol{\theta})$$

A practical parameterization is defined as:

```
python
class DifferentiableSDF(eqx.Module):
    params: jnp.ndarray

    def __call__(self, xyz):
        # Radial basis / kernel representation
        distances = jnp.sqrt(jnp.sum((xyz[:, None, :] -
self.params[None, :, :])**2, axis=-1))
        sdf = distances - 1.0
        return -jax.nn.softplus(-sdf).min(axis=1)
```

B-II.2.2 Differentiable Mask Construction

The SDF is converted into a differentiable mask:

$$m(\mathbf{x}) = H_\epsilon(\phi(\mathbf{x}))$$

```
python
def sdf_mask(sdf_values, epsilon=1e-3):
```

```
        return 0.5 * (1.0 + jnp.tanh(-sdf_values / epsilon))
```

This mask is used consistently across:

- Boundary enforcement (Part II)
- Feature extraction
- Latent solver coupling

B-II.3 Inverse Design Objective Formulation

B-II.3.1 Multi-Objective Loss

The design objective is defined as:

$$\mathcal{L}_{\text{design}} = \alpha L_{\text{drag}} + \beta L_{\text{lift}} + \gamma L_{\text{vorticity}} + \delta L_{\text{smooth}}$$

Where:

L_{drag}	→	drag error
L_{lift}	→	lift error
$L_{\text{vorticity}}$	→	wake structure matching
L_{smooth}	→	geometric regularization

B-II.3.2 Example Numerical Definitions

```
    python
    def compute_drag(u):
        return jnp.mean(u[0])  # placeholder (surface integral in
practice)

    def compute_lift(u):
        return jnp.mean(u[1])

    def vorticity(u, grid):
        return grid.central_diff_x(u[1]) - grid.central_diff_y(u[0])
```

B-II.4 Coupled Flow–Geometry Optimization

B-II.4.1 Single Optimization Step

```
    python
    def inverse_design_step(
```

```
            sdf,
            latent_solver,
            u_history,
            grid,
            dt,
            optimizer,
            targets,
            weights
        ):

            def loss_fn(params):
                sdf.params = params

                # Evaluate SDF mask
                grid_points = jnp.stack(jnp.meshgrid(grid.x, grid.y), axis=-
1).reshape(-1, 2)
                sdf_vals = sdf(grid_points).reshape(grid.nx, grid.ny)
                mask = sdf_mask(sdf_vals)

                # Flow prediction (Part II + III coupling)
                u_final = latent_solver(u_history, dt, grid)

                # Objectives
                drag_loss = (compute_drag(u_final) - targets["drag"])**2
                lift_loss = (compute_lift(u_final) - targets["lift"])**2

                vort = vorticity(u_final, grid)
                vort_loss = jnp.mean((vort - targets["vorticity"])**2)

                smooth_loss = jnp.mean(jnp.gradient(params)**2)

                return (
                    weights["alpha"] * drag_loss +
                    weights["beta"]  * lift_loss +
                    weights["gamma"] * vort_loss +
                    weights["delta"] * smooth_loss
                )

            grads = jax.grad(loss_fn)(sdf.params)
            updates, opt_state = optimizer.update(grads, optimizer.state)
            sdf.params = optax.apply_updates(sdf.params, updates)

            return sdf, opt_state
```

B-II.4.2 Rollout-Based Optimization

```
        python
        def run_inverse_design(
            sdf,
            solver,
            u_init,
            grid,
            dt,
            optimizer,
            targets,
            weights,
            steps=100
        ):
            u_hist = u_init

            for step in range(steps):
                sdf, optimizer.state = inverse_design_step(
                    sdf, solver, u_hist, grid, dt, optimizer, targets, weights
                )
```

```
            u_next = solver(u_hist, dt, grid)
            u_hist = jnp.concatenate([u_hist[1:], u_next[None]], axis=0)

        return sdf
```

B-II.5 Latent Guidance Coupling

The inverse design loop is enhanced using latent predictions:

$$\mathbf{u}_{\text{final}} = \mathcal{S}_{\text{Part II}}(\mathbf{u}) + \mathcal{S}_{\text{latent}}(\mathbf{u})$$

This provides:

- Faster convergence
- Reduced rollout cost
- Improved exploration of feasible flow states

B-II.6 Automated Hyperparameter Optimization

B-II.6.1 Search Space Definition

```
    python
    def define_search_space(trial):
        return {
            "learning_rate": trial.suggest_loguniform("lr", 1e-5, 1e-2),
            "latent_dim": trial.suggest_int("latent_dim", 32, 256),
            "fine_width": trial.suggest_int("fine_width", 64, 512),
            "coarse_width": trial.suggest_int("coarse_width", 64, 512),
            "alpha": trial.suggest_loguniform("alpha", 1e-3, 1.0),
            "beta": trial.suggest_loguniform("beta", 1e-3, 1.0),
            "gamma": trial.suggest_loguniform("gamma", 1e-3, 1.0),
            "delta": trial.suggest_loguniform("delta", 1e-5, 1e-2),
        }
```

B-II.6.2 Objective Function

```
    python
    def objective(trial):

        hparams = define_search_space(trial)

        # Initialize components
        solver = build_solver(hparams)
        sdf = DifferentiableSDF(params=init_params)

        optimizer = optax.adam(hparams["learning_rate"])

        u_hist = init_flow()

        loss_total = 0.0

        for _ in range(rollout_steps):
```

```
            sdf, optimizer.state = inverse_design_step(
                sdf, solver, u_hist, grid, dt, optimizer,
                targets, hparams
            )
            loss_total += evaluate_design(sdf, solver, u_hist)

        return loss_total
```

B-II.7 Multi-Fidelity Training Strategy

B-II.7.1 Curriculum

Training is performed in stages:

1. **Low-resolution pretraining**

   - Small grids
   - Short rollout horizon

2. **Intermediate refinement**

   - Increased latent dimension
   - Longer temporal windows

3. **High-resolution fine-tuning**

   - Full AMR + latent solver
   - Full inverse design loop

B-II.7.2 Implementation

```
    python
    def multi_fidelity_training(n_trials=50):

        study_low = optuna.create_study(direction="minimize")
        study_low.optimize(objective, n_trials=n_trials//2)
        best = study_low.best_params

        # Increase fidelity
        global grid
        grid = high_res_grid()

        study_high = optuna.create_study(direction="minimize")
        study_high.optimize(objective, n_trials=n_trials//2)

        return study_high.best_params
```

B-II.8 Full Training Pipeline

```
    python
    def full_training_pipeline():

        best_hparams = multi_fidelity_training()
```

```
    solver = build_solver(best_hparams)
    sdf = DifferentiableSDF(params=init_params)
    optimizer = optax.adam(best_hparams["learning_rate"])

    u_hist = init_flow()
    for epoch in range(num_epochs):
        sdf, optimizer.state = inverse_design_step(
            sdf, solver, u_hist, grid, dt, optimizer,
            targets, best_hparams)

    return solver, sdf
```

B-II.9 Implementation Notes

- Fully differentiable across all components
- Supports jit, vmap, and distributed execution
- Latent guidance reduces computational cost of optimization
- Multi-fidelity training stabilizes convergence
- Hyperparameter search automates tuning of solver and design system

B-II.10 Summary

Appendix B-II completes the numerical framework by introducing:

- Differentiable geometry representation via SDF
- Fully coupled inverse design optimization
- Latent-guided flow prediction
- Automated hyperparameter tuning
- Multi-fidelity curriculum training
- Together with Appendices B1–B3, this defines a complete, scalable, and fully differentiable pipeline for flow simulation, control, and inverse design.

Appendix B-III: Latent Multi-Fidelity Modeling and Differentiable Control Integration

---

B-III.1 Overview

This appendix presents the latent-space extension of the differentiable hybrid Navier–Stokes framework, enabling:

- High-dimensional flow compression via learned latent representations
- Efficient large-scale and 3D flow simulation
- Multi-fidelity modeling across spatial and temporal scales
- Integration with control and inverse design pipelines

The latent formulation reduces computational complexity while preserving essential flow physics and maintaining full differentiability.

B-III.2 Latent Representation of Flow Fields

B-III.2.1 Encoder Architecture

A convolutional autoencoder (CAE) is used to map physical velocity fields into a compressed latent space:

```
python
class CAELatentEncoder(eqx.Module):
    convs: list

    def __init__(self, key):
        k1, k2, k3 = jax.random.split(key, 3)
        self.convs = [
            eqx.nn.Conv3d(3, 32, 3, stride=2, key=k1),
            eqx.nn.Conv3d(32, 64, 3, stride=2, key=k2),
            eqx.nn.Conv3d(64, 64, 3, stride=2, key=k3),
        ]

    def __call__(self, u):
        x = u.transpose(3, 0, 1, 2)  # [C,H,W,D]
        for conv in self.convs:
            x = jax.nn.relu(conv(x))
        return x
```

B-III.2.2 Decoder Architecture

The latent representation is mapped back to physical space via a transposed convolutional decoder:

```
python
```

```
class CAELatentDecoder(eqx.Module):
    deconvs: list

    def __init__(self, key):
        k1, k2, k3 = jax.random.split(key, 3)
        self.deconvs = [
            eqx.nn.ConvTranspose3d(64, 64, 3, stride=2, key=k1),
            eqx.nn.ConvTranspose3d(64, 32, 3, stride=2, key=k2),
            eqx.nn.ConvTranspose3d(32, 3, 3, stride=2, key=k3),
        ]

    def __call__(self, z):
        x = z
        for deconv in self.deconvs[:-1]:
            x = jax.nn.relu(deconv(x))
        x = self.deconvs[-1](x)
        return x.transpose(1, 2, 3, 0)  # [H,W,D,C]
```

B-III.2.3 Latent Space Properties

The latent representation:

- Compresses high-resolution flow fields
- Preserves dominant vortical and structural features
- Enables efficient learning of non-local interactions.

B-III.3 Latent Dynamics Operator

B-III.3.1 Physics-Informed Latent Update

A latent evolution operator approximates flow dynamics:

```
python
def latent_update(z):
    return -jnp.gradient(z, axis=(1, 2, 3))
```

B-III.3.2 Interpretation

This operator represents:

- Diffusive smoothing in latent space
- A placeholder for structured learned operators
- A bridge between physical PDE dynamics and learned representations.

B-III.4 Latent Solver Integration

B-III.4.1 Hybrid Latent–Physical Solver

```
python
class LatentSolver(eqx.Module):
    encoder: eqx.Module
    decoder: eqx.Module
    fine_nn: eqx.Module
    temporal_nn: eqx.Module
    boundary: eqx.Module
    amr: eqx.Module

    def __call__(self, u_hist, dt, grid):

        # Encode latest state
        z = self.encoder(u_hist[-1])

        # Latent update
        z_next = z + dt * latent_update(z)

        # Decode to physical space
        u_pred = self.decoder(z_next)

        # Update temporal history
        u_hist_new = jnp.concatenate([u_hist, u_pred[None]], axis=0)[-
5:]

        # Apply Part II solver
        return timestep_partII(
            u_hist_new,
            dt,
            grid,
            self.fine_nn,
            self.temporal_nn,
            self.boundary,
            self.amr
        )
```

B-III.4.2 Coupling Mechanism

The latent solver:

- Provides a predictive prior for flow evolution.
- Reduces computational cost of high-resolution updates.
- Interfaces seamlessly with Part II spatial operators

B-III.5 Multi-Fidelity Modeling

B-III.5.1 Hierarchical Representation

The framework supports multiple fidelity levels:

- Low-resolution latent predictions
- Intermediate AMR-enhanced physical fields
- High-resolution corrected solutions

B-III.5.2 Cross-Fidelity Consistency

Consistency is maintained through:

- Shared latent representations.
- Reconstruction losses
- Coupled training objectives.

B-III.6 Differentiable Control and System Integration

B-III.6.1 Control Pathways

The latent solver integrates with control mechanisms through:

- Feature extraction from latent and physical states
- Temporal modeling via transformer-based operators
- Boundary-aware corrections

B-III.6.2 Differentiability

All components remain differentiable across:

- Latent encoding and decoding
- Temporal updates
- Spatial correction operators
- Boundary enforcement

This enables gradient-based optimization of:

- Flow states
- Geometry
- Control inputs

B-III.7 Computational Advantages

The latent formulation provides:

- Reduced dimensionality for large-scale simulations
- Improved scalability to 3D problems
- Efficient long-horizon rollouts
- Compatibility with accelerator hardware

B-III.8 Implementation Notes

- Fully compatible with JAX transformations (jit, vmap, pmap)
- Latent dynamics operator can be replaced with learned neural operators.
- Encoder/decoder architectures may be extended to hierarchical or transformer-based designs.
- Integrates directly with Appendix B-I and B-II pipelines.

B-III.9 Summary

This appendix introduces the latent multi-fidelity extension of the solver, consisting of:

- Learned compression via convolutional encoders.
- Latent-space evolution operators
- Hybrid latent–physical solver integration
- Multi-fidelity consistency across scales

Together with Appendices B-I and B-II, this completes a unified framework for:

- Differentiable flow simulation
- Scalable high-dimensional modeling
- Integrated control and optimization

Appendix C-I: Transition Plan: Baseline Solver to Part I Completion

Project: Differentiable VKVS-Based Flow Framework

Phase: Part I – Baseline Solver Development and Validation

Prepared By: Arno Meijer

Date: 27 March 2026

---

1. Objective

The purpose of this phase is to establish a validated, high-fidelity 2D incompressible Navier–Stokes solver capable of generating vortex shedding phenomena (Von Kármán vortex streets). This will serve as the foundation for later latent acceleration and differentiable inverse design modules.

Key deliverables for Part I include:

1. A functional 2D VKVS solver.
2. Validation against analytical and experimental benchmarks (drag, lift, Strouhal number).
3. Real-time visualization and diagnostic routines using PyQtGraph for high-speed rendering.

---

2. Scope

Included:

- Implementation of velocity–vorticity formulation.
- Spatial discretization using finite difference or spectral schemes.
- Time integration (explicit Runge–Kutta or semi-implicit).
- Boundary condition handling for bluff-body geometries.
- Divergence-free enforcement and energy monitoring.
- Real-time visualization routines with PyQtGraph.

Excluded:

- Latent-space acceleration (Part II).
- Differentiable shape optimization (Part III).

---

3. Assumptions and Requirements

- Solver will be implemented in JAX for automatic differentiation compatibility.

- Initial validation will focus on 2D flows around a circular cylinder.
- Computational domain: uniform Cartesian grid.
- Flow regimes: laminar to transitional (Re = 50–200).
- Outputs required: velocity, vorticity, pressure proxy, drag/lift coefficients, energy metrics.
- Visualizations will prioritize speed and responsiveness using PyQtGraph.

---

4. Transition Plan Overview

The transition from a conceptual baseline solver to Part I completion consists of five major stages:

1. Grid and Domain Setup
2. Velocity–Vorticity Solver Implementation
3. Boundary Conditions and Obstacle Handling
4. Time Integration and Stability Checks
5. Validation and Visualization with PyQtGraph

---

5. Detailed Steps and Pseudo-Code

5.1 Grid and Domain Setup

Objective: Define computational grid, boundary layers, and initial conditions.

Actions:

- Define domain size $L_x \times L_y$ and resolution $N_x \times N_y$
- Initialize velocity field u, v to zero or small perturbations
- Initialize vorticity field ω from velocity

Pseudo-Code:

```
python
# Define grid
Nx, Ny = 200, 100
Lx, Ly = 10.0, 5.0
dx, dy = Lx / Nx, Ly / Ny
x = linspace(0, Lx, Nx)
y = linspace(0, Ly, Ny)

# Initialize velocity and vorticity
u = zeros(Nx, Ny)
v = zeros(Nx, Ny)
omega = compute_vorticity(u, v)
```

5.2 Velocity–Vorticity Solver Implementation

Objective: Solve the incompressible Navier–Stokes equations using vorticity transport and velocity recovery.

Actions:

- Implement vorticity transport equation:

$$\frac{\partial \omega}{\partial t} + \mathbf{u} \cdot \nabla \omega = \nu \nabla^2 \omega$$

- Recover velocity via streamfunction ψ:

$$\nabla^2 \psi = -\omega, u = \frac{\partial \psi}{\partial y}, v = -\frac{\partial \psi}{\partial x}$$

Pseudo-Code:

```
python
def advect_vorticity(omega, u, v, dt):
    # Compute convective term using upwind or central difference
    conv = u * d_dx(omega) + v * d_dy(omega)
    diff = nu * laplacian(omega)
    omega_new = omega + dt * (-conv + diff)
    return omega_new

def velocity_from_streamfunction(omega):
    psi = poisson_solve(-omega)  # Solve ∇²ψ = -ω
    u = d_dy(psi)
    v = -d_dx(psi)
    return u, v
```

5.3 Boundary Conditions and Obstacle Handling

Objective: Accurately impose no-slip conditions and obstacle constraints.

Actions:

- Define obstacle mask for cylinder
- Enforce zero velocity inside obstacle and adjust vorticity at boundaries

```
Pseudo-Code:
python
# Define cylinder mask
cx, cy, r = Lx/4, Ly/2, 0.5
mask = (X - cx)**2 + (Y - cy)**2 <= r**2

# Apply no-slip BC
u[mask] = 0.0
v[mask] = 0.0
```

```
# Update vorticity at obstacle boundaries
omega = update_vorticity_boundaries(u, v, omega, mask)
```

5.4 Time Integration and Stability Checks

Objective: Integrate the solver in time with controlled stability.

Actions:

- Use explicit Runge–Kutta 4th order (RK4) or semi-implicit scheme
- Compute CFL condition to adapt dt
- Monitor divergence and kinetic energy for diagnostics

Pseudo-Code:

```
python
for step in range(N_steps):
    dt = compute_cfl(u, v, dx, dy)
    omega = advect_vorticity(omega, u, v, dt)
    u, v = velocity_from_streamfunction(omega)

    # Diagnostics
    div = compute_divergence(u, v)
    energy = compute_kinetic_energy(u, v)
    log(step, div, energy)
```

5.5 Validation and Visualization with PyQtGraph

Objective: Confirm solver fidelity and generate high-speed real-time visualizations.

Actions:

- Compute drag/lift coefficients and Strouhal number for validation
- Visualize vorticity and velocity magnitude using PyQtGraph's ImageView or PlotWidget
- Ensure fast refresh and interactive controls for domain exploration

Pseudo-Code:

```
python
import pyqtgraph as pg
from pyqtgraph.Qt import QtGui

app = QtGui.QApplication([])

# Initialize PyQtGraph ImageView for vorticity
vorticity_view = pg.ImageView()
vorticity_view.show()

for step in range(N_steps):
    dt = compute_cfl(u, v, dx, dy)
    omega = advect_vorticity(omega, u, v, dt)
    u, v = velocity_from_streamfunction(omega)
```

```
            # Update PyQtGraph visualization
            vorticity_view.setImage(omega.T)  # transpose for correct
orientation

        # Drag/lift computations
        drag, lift = compute_forces(u, v, mask)
        st_number = compute_strouhal(lift, dt)
```

---

6. Acceptance Criteria for Part I

- Solver produces stable vortex shedding for Re ≥ 100
- Divergence remains $< 1 \times 10^{-5}$ for all timesteps
- Drag, lift, and Strouhal number within 5–10% of benchmark values
- PyQtGraph visualizations render in real-time and update smoothly

---

7. Dependencies and Resources

- JAX or equivalent numerical library for automatic differentiation and GPU acceleration
- Poisson solver (spectral or iterative) for streamfunction recovery
- PyQtGraph for high-performance, interactive visualizations

Appendix C-II: Transition Plan: Part I to Part II Completion

Project: Differentiable VKVS-Based Flow Framework

Phase: Part II – Latent-Space Acceleration and Neural Solver Integration

Prepared By: Arno Meijer

Date: 27 March 2026

---

1. Objective

This phase introduces a latent-space neural solver that accelerates the 2D VKVS baseline developed in Part I. The neural operator learns coarse-scale flow patterns while maintaining incompressibility and energy conservation, enabling real-time updates and interactive design workflows.

Key deliverables for Part II include:

1. A trained latent-space neural operator capable of predicting flow evolution.
2. Integration of the neural solver with Part I's VKVS framework.
3. Real-time latent-space visualization using PyQtGraph for monitoring neural predictions.

---

2. Scope

Included:

- Construction of a latent encoder–decoder architecture.
- Neural operator training on Part I flow data.
- Integration of latent solver with full VKVS solver.
- Real-time visualization of latent predictions and reconstruction fields.

Excluded:

- Differentiable inverse design (Part III).
- 3D flow extension.

---

3. Assumptions and Requirements

1. Latent solver implemented in JAX for GPU acceleration and automatic differentiation.

2. Training data generated from Part I simulations with varied initial conditions.
3. Latent dimension chosen to balance speed and fidelity.
4. PyQtGraph used to visualize both latent vectors and reconstructed physical fields.
5. Neural operator must preserve divergence-free condition.

---

4. Transition Plan Overview

The transition from Part I to Part II consists of four major stages:

1. Data Preparation and Latent Space Definition
2. Neural Operator Architecture Design
3. Training and Validation
4. Integration with Part I Solver and Visualization

---

5. Detailed Steps and Pseudo-Code

5.1 Data Preparation and Latent Space Definition

Objective: Prepare Part I simulation outputs for neural operator training.

Actions:

- Collect velocity and vorticity fields at selected timesteps
- Flatten or reshape fields for latent encoder input
- Normalize inputs for neural stability

Pseudo-Code:

```
       python
       # Extract training snapshots
       snapshots = []
       for step in range(0, N_steps, snapshot_interval):
           snapshots.append(concatenate([u[step], v[step], omega[step]],
axis=0))

       snapshots = stack(snapshots)  # Shape: (num_snapshots, Nx, Ny,
channels)

       # Normalize
       snapshots = snapshots / max(abs(snapshots))
       Define latent dimension:
               latent_dim = 64 (adjustable for speed vs accuracy)
```

5.2 Neural Operator Architecture Design

Objective: Build an encoder-decoder latent neural operator.

Actions:

- Encoder compresses full flow field to latent vector

- Decoder reconstructs velocity/vorticity fields from latent vector
- Neural operator predicts next latent state given current latent state

Pseudo-Code:

```
python
# Encoder
latent = MLP_Encoder(input_shape=(Nx*Ny*channels),
latent_dim=latent_dim)

# Latent update (neural operator)
latent_next = LatentOperator(latent)

# Decoder
u_recon, v_recon, omega_recon = MLP_Decoder(latent_next)
Loss function preserves divergence-free and energy:
```

$$loss = mse(u_{recon}, u_{target}) + mse(v_{recon}, v_{target}) + divergence_penalty(u_{recon}, v_{recon}) + energy_penalty(u_{recon}, v_{recon})$$

5.3 Training and Validation

Objective: Train the neural operator to predict latent evolution accurately.

Actions:

- Use Part I snapshots as training data
- Apply gradient-based optimization with JAX
- Validate on held-out flow sequences

Pseudo-Code:

```
python
for epoch in range(num_epochs):
    for batch in data_loader(snapshots):
        latent = Encoder(batch[:-1])
        latent_pred = LatentOperator(latent)
        recon_pred = Decoder(latent_pred)

        loss = compute_loss(recon_pred, batch[1:])
        grads = jax.grad(loss, params)
        update_params(params, grads)

    # Validation
    val_loss =
compute_loss(Decoder(LatentOperator(Encoder(val_batch[:-1]))),
val_batch[1:])
    log(epoch, loss, val_loss)
```

5.4 Integration with Part I Solver and Visualization

Objective: Replace coarse time evolution in Part I solver with latent predictions while maintaining fidelity.

Actions:

- Compute latent vector from Part I solver state
- Use latent operator to predict next state
- Reconstruct full fields and update Part I solver
- Visualize latent space evolution and reconstructed flow using PyQtGraph

Pseudo-Code:

```
        python
        import pyqtgraph as pg
        from pyqtgraph.Qt import QtGui

        app = QtGui.QApplication([])
        latent_view = pg.ImageView(title="Latent Magnitude")
        flow_view = pg.ImageView(title="Reconstructed Vorticity")
        latent_view.show()
        flow_view.show()

        # Main loop
        for step in range(N_steps):
            # Encode Part I state
            latent = Encoder(concatenate([u, v, omega]))

            # Predict latent evolution
            latent_next = LatentOperator(latent)

            # Decode to physical field
            u_recon, v_recon, omega_recon = Decoder(latent_next)

            # Update solver fields
            u, v, omega = u_recon, v_recon, omega_recon

            # Visualization
            latent_view.setImage(abs(latent_next).reshape(sqrt(latent_dim),
sqrt(latent_dim)).T)
            flow_view.setImage(omega_recon.T)
        Ensure divergence-free enforcement post-reconstruction:
        python
        u, v = project_to_divergence_free(u, v)
```

6. Acceptance Criteria for Part II

- Latent operator predicts flow evolution with $<5\%$ error in velocity/vorticity fields
- Divergence remains $< 1 \times 10^{-5}$ after latent reconstruction
- Real-time PyQtGraph visualizations of latent and reconstructed fields run at interactive speeds (> 20 FPS)
- Coarse flow features (vortex shedding, drag/lift trends) match Part I baseline

---

7. Dependencies and Resources

- JAX or equivalent for neural operator and auto-differentiation
- Pre-processed Part I simulation data for training
- PyQtGraph for interactive visualization of latent vectors and reconstructed fields
- GPU acceleration recommended for real-time latent updates

Appendix C-III: Transition Plan: Part II to Part III Completion

Project: Differentiable VKVS-Based Flow Framework

Phase: Part III – Differentiable Inverse Design

Prepared By: Arno Meijer

Date: 27 March 2026

---

1. Objective

This phase implements differentiable inverse design using the latent-space solver developed in Part II. The goal is to morph geometry in real-time to achieve target flow objectives (e.g., vorticity distribution, drag reduction) while preserving physical constraints.

Key deliverables:

1. Differentiable SDF-based geometry representation linked to the latent solver.
2. Real-time geometry optimization via gradient-based updates.
3. PyQtGraph visualizations for geometry, flow, and latent-space evolution.

---

2. Scope

Included:

- Signed Distance Function (SDF) for geometry representation.
- Coupling latent-space solver with SDF gradients.
- Gradient-based optimization to reach target flow objectives.
- Interactive PyQtGraph visualization of geometry morphing.

Excluded:

- Extension to 3D flows (future work).
- External CFD solvers outside latent-space framework.

---

3. Assumptions and Requirements

1. Geometry represented as a differentiable SDF.
2. Latent solver from Part II is pre-trained and differentiable.
3. Target flow objectives are defined as field-based loss functions.

4. PyQtGraph used to visualize geometry and flow in real-time.
5. Optimization loop runs with stability constraints to prevent unphysical geometries.

---

4. Transition Plan Overview

Transition from Part II to Part III involves three major stages:

1. Geometry Representation and Coupling
2. Inverse Design Optimization Loop
3. Visualization and Monitoring

---

5. Detailed Steps and Pseudo-Code

5.1 Geometry Representation and Coupling

Objective: Represent geometry in a differentiable format that the latent solver can interact with.

Actions:

- Implement geometry as SDF
- Sample geometry at solver grid points
- Compute signed distance gradients for differentiable flow–geometry interaction

Pseudo-Code:

```
python
# Define SDF grid
sdf = initialize_sdf(Nx, Ny)

# Sample geometry at solver points
def sdf_sample(x, y):
    return sdf[x, y]

# Compute gradients for differentiable updates
sdf_grad_x, sdf_grad_y = jax.grad(sdf_sample, argnums=(0,1))
Couple geometry to latent solver:
python
# Modify latent update based on geometry influence
latent_next = LatentOperator(latent, sdf_sample(grid_x, grid_y))
```

5.2 Inverse Design Optimization Loop

Objective: Update geometry to minimize flow objectives using differentiable latent solver predictions.

Actions:

- Define target objective (e.g., minimize vorticity in a region, match target flow)

- Compute differentiable loss between predicted and target flow
- Backpropagate loss to SDF and update geometry

```
Pseudo-Code:
python
# Target flow field
target_flow = load_target_flow()

# Optimization loop
for iter in range(max_iters):
    # Encode current flow
    latent = Encoder(concatenate([u, v, omega]))

    # Predict next state via latent solver with current geometry
    latent_next = LatentOperator(latent, sdf)
    u_pred, v_pred, omega_pred = Decoder(latent_next)

    # Compute differentiable loss
    flow_loss = mse(u_pred, target_flow.u) + mse(v_pred,
target_flow.v)
    flow_loss += mse(omega_pred, target_flow.omega)

    # Add regularization for geometry smoothness
    geom_loss = smoothness_penalty(sdf)
    total_loss = flow_loss + lambda_geom * geom_loss

    # Compute gradients w.r.t SDF
    grads = jax.grad(total_loss, sdf)

    # Update SDF
    sdf -= learning_rate * grads

    # Optional: enforce physical constraints
    sdf = enforce_bounds(sdf)
```

5.3 Visualization and Monitoring

Objective: Display geometry morphing and flow evolution in real-time.

Actions:

- Use PyQtGraph to show geometry SDF, latent magnitude, and reconstructed flow
- Update visualizations each optimization iteration

Pseudo-Code:

```
python
import pyqtgraph as pg
from pyqtgraph.Qt import QtGui

app = QtGui.QApplication([])
geom_view = pg.ImageView(title="SDF Geometry")
latent_view = pg.ImageView(title="Latent Magnitude")
flow_view = pg.ImageView(title="Reconstructed Vorticity")

geom_view.show()
latent_view.show()
flow_view.show()

for iter in range(max_iters):
    # ... compute u_pred, v_pred, omega_pred, sdf ...
```

```
            # Visualization updates
            geom_view.setImage(sdf.T)
            latent_view.setImage(abs(latent_next).reshape(sqrt(latent_dim),
sqrt(latent_dim)).T)
            flow_view.setImage(omega_pred.T)
        Optional: save frames to create an animation for documentation or
LinkedIn demonstrations:
        python
        if iter % save_interval == 0:
            save_image(geom_view, f"geom_{iter}.png")
            save_image(flow_view, f"flow_{iter}.png")
```

---

6. Acceptance Criteria for Part III

- Geometry morphs to meet target flow objectives within defined tolerance ($<5\%$ error)
- Latent-space predictions remain consistent and divergence-free throughout optimization
- Real-time PyQtGraph visualization runs at interactive speeds ($>$ 15 FPS)
- Geometry smoothness and physical bounds maintained at all iterations

---

7. Dependencies and Resources

- Part II latent solver (pre-trained)
- JAX for differentiable geometry updates
- PyQtGraph for interactive visualization
- GPU acceleration recommended for real-time inverse design
- Target flow objectives (field data or analytical reference)

Appendix D - Towards a 3D Differentiable Hybrid CFD–ML Framework for High-Performance Super-Clusters

Author: Arno Meijer

Purpose: This document outlines a roadmap for extending the existing 2D differentiable hybrid Navier–Stokes framework (Parts I–III) to fully three-dimensional flows, unstructured meshes, and multi-node GPU clusters using NVLink. The approach also integrates NVIDIA Warp for high-performance differentiable kernel execution to support large-scale inverse design, adaptive resolution, and long-horizon latent rollouts.

---

1. Motivation

The current 2D solver framework demonstrates fully differentiable, multi-scale neural correction, sharp boundary enforcement, latent-space rollouts, and reinforcement-driven inverse design. Scaling to realistic engineering problems requires:

1. Three-dimensional domains for complex aerodynamic and hydrodynamic geometries.
2. Support for unstructured meshes (tetrahedral, polyhedral) to handle arbitrary shapes and boundary layers.
3. Multi-GPU and super-cluster execution to manage memory and computational demands.
4. High-performance differentiable kernels to preserve throughput and accuracy while maintaining end-to-end backpropagation.

---

2. 3D Solver Extension

2.1 Governing Equations

Extend Navier–Stokes equations to 3D:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u} + \nabla \cdot \boldsymbol{\tau}_{\text{SGS}} - \frac{\chi}{\epsilon}\mathbf{u} + \mathbf{f}_{\text{act}}$$

$$\nabla \cdot \mathbf{u} = 0$$

Subgrid-scale (SGS) closures, Brinkman penalization, and differentiable actuation terms remain identical conceptually, but are extended to 3D tensor operations.

2.2 3D Discretization

- **Structured 3D grids:** for rapid prototyping and small-scale inverse design.
- **Unstructured meshes:** tetrahedral, polyhedral, or hybrid to resolve complex boundaries and high aspect-ratio boundary layers.
- Use finite-volume or finite-element discretization with differentiable operators.

2.3 Boundary and Interface Handling

Maintain Brinkman penalization for smooth differentiability.

Employ SDF-based sharp boundary recovery extended to 3D:

$$\mathbf{u}^{\text{final}} = \mathbf{u}^{\text{corr}} \cdot (1 - H_\epsilon(\phi(\mathbf{x}; \boldsymbol{\theta})))$$

---

3. Neural Operator Extensions

3.1 Fine-Scale and Coarse-Scale Operators

- Convolutions become 3D ($3 \times 3 \times 3$ kernels) or graph-based for unstructured data.
- Fine-scale neural operators manage small vortical structures, while coarse-scale operators capture global divergence and pressure redistribution.

3.2 Latent Representations

- Latent dimension may need adjustment ($64\text{D} \rightarrow 128\text{–}256\text{D}$) to accommodate richer 3D flow features.
- Use graph neural operators for unstructured meshes.
- Integrate temporal sequence modeling for long-horizon predictions in 3D flow.

3.3 Reinforcement-Driven Design

- State: $\mathbf{s}_t = \{\mathbf{F}_t, \mathbf{s}(\mathbf{x}, t)\}$, updated with 3D features.
- Action: $\mathbf{a}_t = \{\mathbf{f}_{\text{act}}, \boldsymbol{\theta}\}$, including geometry morphing and control of actuation surfaces.
- Policy optimization continues using gradient-based or hybrid gradient + RL methods.

4. High-Performance GPU Cluster Strategy

4.1 Multi-GPU Parallelization

- Use NVLink for low-latency inter-GPU communication.
- Distribute the domain across nodes with halo exchange for boundary synchronization.
- Latent rollouts and fine-scale kernels executed on individual GPUs, coarse-scale or latent corrections globally via NVLink.

4.2 Warp Integration

Implement 3D advection, diffusion, divergence, curl, and Brinkman penalization kernels in Warp.

Warp kernels allow:

- Memory-optimized 3D stencil computation.
- Differentiable backpropagation through physical operators.
- High throughput on 3D structured or unstructured grids.

4.3 Memory Optimization

- Employ mixed precision (FP16/FP32) for neural operators where stability allows.
- Checkpointing of intermediate states to reduce memory usage for long-horizon gradient propagation.
- Use NVLink for seamless multi-node memory aggregation, enabling 3D grids of tens of millions of cells.

5. Numerical Stability and Accuracy

5.1 Mitigating Brinkman Stiffness

- Implement implicit-explicit (IMEX) schemes: explicit for advection, implicit for stiff Brinkman term.

- Warp kernels can accelerate implicit solves via Jacobi, GMRES, or multi-grid solvers on GPU.

5.2 Boundary Layer Accuracy

- Adaptive 3D SDF resolution with subgrid refinement ensures sharp interface capture for high-Reynolds-number flows.
- Combine with multi-scale neural correction to restore high-frequency flow structures near walls.

5.3 Latent Rollout Stability

- Introduce periodic physical correction steps where latent predictions are projected back onto a physics-consistent field.
- Use spectral regularization in neural operators to counteract high-frequency bias.

---

6. Scalability Considerations

- 3D convolutional operators optimized for Warp to minimize kernel launch overhead.
- Graph-based operators for unstructured meshes allow arbitrary geometry handling while maintaining differentiability.
- NVLink multi-node distribution ensures that memory and computation scale linearly with cluster size.
- Pipeline parallelism: forward physics + neural correction + latent update + gradient backprop executed as asynchronous stages across GPUs.

---

7. Roadmap

1. Prototype 3D structured solver in JAX for differentiability.
2. Integrate Warp kernels for 3D physics operators to accelerate performance.
3. Introduce unstructured mesh support using graph neural operators and differentiable FE/FVM stencils.
4. Develop multi-node execution with NVLink for large-scale grids.
5. Validate on canonical 3D problems: 3D airfoils, cylinder wake, and turbulence benchmarks.
6. Optimize inverse design and latent rollouts with gradient checkpointing and periodic physical projections.

---

8. Expected Outcomes

- Full 3D differentiable hybrid solver capable of handling complex geometries on unstructured meshes.
- High-fidelity inverse design, including geometry morphing, actuation control, and long-horizon latent rollouts.

- Efficient scaling across GPU clusters using NVLink and Warp-accelerated kernels.
- Accurate resolution of boundary layers, vortical structures, and global pressure fields without explicit Poisson solves.

Appendix E - Mitigation Strategies for Numerical and Learning Challenges in a Differentiable Hybrid CFD–ML Solver

Author: Arno Meijer

Purpose: This document outlines key challenges inherent to the differentiable hybrid Navier–Stokes framework and proposes strategies to mitigate numerical, modeling, and learning limitations while preserving differentiability, stability, and design fidelity.

---

1. Numerical Stiffness from Brinkman Penalization

Issue:

The Brinkman penalization term,

$$-\frac{\chi(\mathbf{x};\boldsymbol{\theta})}{\epsilon}\mathbf{u}$$

introduces stiffness when ϵ is small, forcing overly restrictive timesteps if treated explicitly.

Mitigation Strategy:

1. **Implicit–Explicit (IMEX) Scheme:** Treat Brinkman term implicitly while keeping the remaining advection–diffusion terms explicit. This allows larger timesteps while maintaining differentiability.
2. **Adaptive Permeability Scheduling:** Gradually reduce ϵ during simulation or optimization. Start with a higher permeability to accelerate initial convergence, then anneal toward physical limits for final boundary fidelity.
3. **Sub-stepping for Penalized Regions:** Apply fine temporal sub-steps only inside solid/masked regions while keeping the main fluid domain larger.

Expected Outcome: Improved stability without fully reverting to computationally expensive global implicit solvers.

---

2. Boundary Layer "Smearing"

Issue:

Brinkman penalization and differentiable masking create a diffuse boundary, which blurs velocity gradients. High-Re flows are sensitive to near-wall accuracy, impacting drag and lift calculations.

Mitigation Strategy:

1. **Sharp SDF-Based Recovery:** Apply Part II's Signed Distance Function (SDF) masking to reinforce velocity suppression near walls.
2. **Gradient-Aware Loss Weighting:** Penalize deviations of near-wall velocity gradients in loss function:

$$\mathcal{L}_{\text{wall}} = \sum_{\mathbf{x} \in \text{boundary}} \| \nabla \mathbf{u} - \nabla \mathbf{u}_{\text{target}} \|^2$$

Expected Outcome: Restoration of near-wall accuracy while retaining smoothness for gradient propagation.

---

3. Accumulated Error in Latent Rollouts

Issue:

Latent space rollouts drift over long horizons due to small errors in the compressed representation.

Mitigation Strategy:

1. **Periodic Physical Reprojection:** After N latent steps, decode and perform a divergence-free correction using the physical grid solver.
2. **Latent Correction Loss:** Include a divergence and vorticity consistency term within latent space training:

$$\mathcal{L}_{\text{latent-correct}} = \| \nabla \cdot \mathcal{D}(\mathbf{z}) \|^2 + \| \omega - \omega_{\text{target}} \|^2$$

Expected Outcome: Stabilized latent rollouts and prevention of unphysical explosion over long horizons.

---

4. Spectral Bias in Neural Operators

Issue:

Neural networks preferentially learn low-frequency modes, potentially missing small-scale eddies critical for energy dissipation.

Mitigation Strategy:

1. **Fourier Feature Encoding:** Preprocess input features with sinusoidal positional encodings to enhance high-frequency learning.
2. **Multi-Scale Loss:** Include scale-weighted losses emphasizing both low- and high-wavenumber components.
3. **Hybrid Correction Layers:** Use fine-scale physics-based SGS operators to capture unresolved high-frequency content.

Expected Outcome: Better representation of turbulent scales and energy cascades.

---

5. Conservation Law Violations in Latent Space

Issue:

Latent Dynamics Operator may violate mass or energy conservation if left unconstrained.

Mitigation Strategy:

1. **Physics-Constrained Latent Operator:** Add divergence and kinetic energy penalties directly in latent training.
2. **Constrained Decoder Architecture:** Enforce curl-based, divergence-preserving reconstruction, with spectral filtering to prevent "shortcut" solutions.
3. **Soft Penalty Regularization:** Introduce Lagrange-multiplier style terms for global conservation constraints.

Expected Outcome: Latent predictions respect fundamental Navier–Stokes invariants, reducing unphysical solutions.

---

6. Vanishing Gradients in Inverse Design

Issue:

Backpropagating through long-horizon simulations can cause vanishing or exploding gradients, hindering stable geometric morphing.

Mitigation Strategy:

1. **Gradient Checkpointing:** Save intermediate activations selectively to reduce memory cost while preserving gradient accuracy.
2. **Temporal Truncation / Windowing:** Break horizon into overlapping windows, performing backpropagation within each window to stabilize gradients.
3. **Normalized Updates:** Rescale gradients for geometry updates to prevent numerical instability during long sequences.

Expected Outcome: Reliable inverse design updates across thousands of timesteps.

---

7. Inaccurate SDF Gradient Sampling

Issue:

Coarse solver grids yield noisy SDF gradients, producing jagged geometry updates.

Mitigation Strategy:

1. **Super-sampled Gradient Interpolation:** Sample SDF at higher resolution and interpolate gradients onto solver grid.
2. **Smooth Regularization:** Apply total variation or Laplacian smoothing to the gradient field used for geometry optimization.
3. **Adaptive Mesh Alignment:** Align SDF sampling points with regions of high flow sensitivity (e.g., leading/trailing edges).

Expected Outcome: Smooth and accurate geometric updates with well-resolved gradients.

---

8. Training Data Generalization

Issue:

Latent Operator trained on Part I simulations may fail for out-of-distribution geometries in Part III.

Mitigation Strategy:

1. **Data Augmentation:** Include geometries with parameter perturbations, rotations, and scaling to improve latent generality.
2. **Curriculum Training:** Gradually expose the latent solver to increasingly different geometries during training.
3. **Hybrid Physical Checkpoints:** Perform occasional physical solver rollouts on new geometries to correct latent deviations.

Expected Outcome: Robust latent operator capable of handling novel shapes during optimization.

---

9. Curse of Dimensionality in 3D

Issue:

Extending to 3D imposes heavy memory and compute demands due to 3D convolutions and latent reconstruction.

Mitigation Strategy:

1. **Separable 3D Convolutions:** Reduce memory footprint using factorized 3D convolutions.
2. **Hybrid 2.5D Slices:** Process flow as a sequence of 2D slices with inter-slice coupling.
3. **Latent Compression:** Further compress latent space for 3D while maintaining divergence-preserving reconstruction.

Expected Outcome: Maintain high FPS performance for 3D simulations on consumer GPUs.

---

10. Soft Boundary Interface Stiffness

Issue:

Sharpening the Heaviside mask for boundary recovery introduces gradient stiffness, causing oscillatory or unstable geometry optimization.

Mitigation Strategy:

1. **Gradual Mask Annealing:** Slowly increase mask sharpness during optimization, allowing the system to adapt smoothly.
2. **Stability-Weighted Updates:** Scale geometry gradient updates near high-stiffness regions to prevent overshoot.
3. **Regularized SDF Loss:** Include smoothness penalty for geometric updates:

$$\mathcal{L}_{\text{SDF-smooth}} = \| \nabla^2 \phi \|^2$$

Expected Outcome: Sharp boundaries achieved without destabilizing the differentiable optimization loop.

Conclusion

By systematically combining physics-aware corrections, latent stabilization, adaptive sampling, and gradient-aware optimization strategies, this framework addresses the key challenges of numerical stiffness, latent drift, boundary fidelity, spectral bias, and long-horizon differentiable design. These strategies ensure that the solver maintains physical fidelity, computational efficiency, and differentiable inverse design capabilities, while remaining extendable to 3D and high-Reynolds-number flows.